

Key-Value Store v8

Versioni e compare-and-set

Architetture dei Sistemi Distribuiti

Lezione 17

Perche' Non Basta SET

Finora il client poteva fare due cose:

- leggere un valore;
- sovrascriverlo con SET.

Ma SET e' una *blind write*:

- non dice quale stato il client avesse osservato;
- non protegge da aggiornamenti concorrenti;
- non distingue “scrittura legittima” da “scrittura su stato ormai vecchio”.

La domanda di fondo e':

il client puo' ancora fidarsi dello stato che aveva letto prima di scrivere?

Se la risposta e' "forse no", serve un contratto piu' ricco:

- il server deve esporre uno stato osservato;
- il client deve poter subordinare la scrittura a quello stato;
- il fallimento deve essere semantico, non solo tecnico.

Lost Update Senza CAS

Senza una scrittura condizionale, una storia del genere e' possibile:

Client A: GET course -> ads

Client B: GET course -> ads

Client A: SET course systems

Client B: SET course networks

L'ultimo writer vince, ma il sistema non dice nulla sul conflitto. Quello che manca non e' solo protezione interna: manca un modo per rendere il conflitto visibile all'interfaccia.

La lezione introduce due comandi:

- GETV <key>: restituisce valore e versione;
- CAS <key> <expected_version> <value>: scrive solo se la versione e' quella attesa.

Da questo momento la versione:

- non e' piu' solo metadato interno;
- entra nel linguaggio del client;
- diventa parte del contratto dell'API.

```
SET course ads  
-> OK version=0
```

```
GETV course  
-> OK ads version=0
```

```
CAS course 0 advanced-distributed-systems  
-> OK version=1
```

Interpretazione:

- il client ha osservato la versione 0;
- il server conferma che nulla e' cambiato nel frattempo;
- la scrittura viene accettata e la versione avanza.

Client A: GETV course -> OK ads version=0

Client B: GETV course -> OK ads version=0

Client A: CAS course 0 v1 -> OK version=1

Client B: CAS course 0 v2 -> ERR version_mismatch current=1

Questo errore non e' rumore di rete. E' il contratto che dice:

- la precondizione dichiarata dal client non e' piu' vera;
- quindi la scrittura non e' autorizzata.

Con CAS il client corretto non esegue piu' una singola write. Esegue un piccolo protocollo:

- ① osserva stato e versione;
- ② prepara una decisione applicativa;
- ③ tenta la scrittura condizionale;
- ④ in caso di mismatch, riapre il ciclo.

Il server delega quindi una parte del controllo di concorrenza al client, ma lo fa in modo esplicito e disciplinato.


```
_, current_version = self._data.get(key, ("", -1))  
next_version = current_version + 1  
self._data[key] = (value, next_version)
```

- una chiave nuova parte implicitamente da versione -1;
- il primo SET produce versione 0;
- ogni aggiornamento incrementa monotonicamente il contatore locale.

Qui emerge già' una scelta di contratto: la versione e' per-chiave, non globale all'intero store.

```
record = self._data.get(key)
current_version = -1 if record is None else record[1]
if current_version != expected_version:
    return f"ERR version_mismatch current={current_version}"
next_version = current_version + 1
self._data[key] = (value, next_version)
```

La logica importante e' tutta dentro una sezione critica:

- confronto della versione;
- decisione di accettare o rifiutare;
- eventuale aggiornamento.

Per una chiave possiamo immaginare stati come:

- assente, versione implicita -1 ;
- presente con valore v e versione n ;
- aggiornata a $n+1$ dopo una write accettata.

Transizioni rilevanti:

- SET: forza una transizione allo stato successivo;
- CAS valida: transizione condizionata;
- CAS invalida: nessun cambiamento di stato.

Con GETV e CAS vogliamo ottenere almeno queste proprietà:

- una CAS non può sovrascrivere silenziosamente un update già avvenuto;
- un client con osservazione stantia viene respinto esplicitamente;
- il server produce una storia locale coerente delle versioni di una chiave.

Non stiamo ancora garantendo serializzabilità globale. Stiamo però rendendo osservabile il conflitto.

Il contratto migliora la safety, ma introduce un costo:

- alcune scritture ora possono fallire legittimamente;
- il client deve gestire retry o nuova lettura;
- il sistema puo' progredire solo se i client rispettano il protocollo.

Questa e' una tensione classica:

- piu' informazione semantica all'interfaccia;
- meno comodita' per il chiamante;
- maggiore controllo sui conflitti.

Che Cosa Fa Davvero L'Implementazione

Nel lab il server conserva per ogni chiave:

`(value, version)`

Le operazioni importanti sono:

- SET: incrementa sempre la versione;
- CAS: confronta la versione attesa con quella corrente;
- se il confronto riesce, aggiorna valore e versione nella stessa sezione critica.

Questa e' la soluzione minima corretta su nodo singolo.

Lock globale

- semplice;
- ottimo per fissare la semantica;
- serializza piu' del necessario.

Lock per chiave

- stessa semantica pubblica;
- piu' parallelismo;
- maggiore complessita' interna.

Storage atomico sottostante

- il server espone il contratto;
- l'atomicita' viene delegata a un motore piu' ricco.

Per il client, un update con CAS richiede spesso:

- ➊ GETV;
- ➋ decisione applicativa locale;
- ➌ CAS;
- ➍ eventualmente nuova lettura e retry.

Quindi il costo cresce con:

- latenza di rete;
- frequenza dei conflitti;
- numero di retry necessari.

CAS non elimina il conflitto. Lo rende esplicito. Un client serio deve decidere:

- se rileggere sempre dopo mismatch;
- quante volte riprovare;
- se introdurre backoff;
- quando fermarsi e segnalare fallimento all'applicazione.

Senza questa disciplina possono apparire:

- loop aggressivi;
- spreco di round trip;
- starvation di alcuni client.

Cosa Deve Fare Un Client Ben Progettato

Un client che usa davvero CAS dovrebbe:

- distinguere mismatch da errore di rete;
- evitare retry ciechi con la stessa versione;
- rileggere lo stato quando la precondition non vale piu';
- decidere a livello applicativo se riprovare, unire o abortire.

Questa parte e' importante: CAS non elimina il conflitto. Lo rende governabile.

Se esponiamo una versione, dobbiamo specificare bene:

- la versione è per chiave o globale?
- DELETE cambia la storia della chiave o la resetta?
- quale versione corrisponde a una chiave assente?
- il valore di `current=` in un mismatch è informativo o normativo?

Un contratto vago qui genera client fragili e implementazioni incompatibili.

La chiave assente e' un caso didatticamente utile.

- nel lab e' modellata come versione implicita -1;
- quindi una CAS `key -1 value` puo' essere letta come "create-if-absent";
- questa e' gia' una scelta semantica forte, non un dettaglio banale.

Se cambiamo questa convenzione, cambiamo il significato dell'interfaccia.

Perche' Questo Serve Anche Nelle Tappe Dopo

Quando una chiave verra':

- replicata;
- migrata tra shard;
- osservata da client diversi in momenti diversi;

la domanda non sara' piu' solo "qual e' il valore?" ma anche:

- quale versione e' stata letta?
- quella versione e' ancora valida nel nuovo contesto?
- chi garantisce la monotonicita' della storia osservata?

Perche' Questa Tappa E' Strategica

CAS e' un ponte tra piu' problemi del corso:

- concorrenza locale;
- replica con conflitti;
- migrazione tra shard;
- coordinamento client-side basato su stato osservato.

Se non fissiamo bene il significato della versione ora, nelle tappe successive ci trasciniamo ambiguita' molto piu' costose.

Con GETV e CAS il server non promette piu' soltanto:

- “questo e' il valore”;
- “questa scrittura e' riuscita”.

Promette anche:

- quale stato il client ha osservato;
- sotto quale preconditione accetta una scrittura;
- come segnala che quella preconditione e' diventata falsa.